AIMERZ.ai
Aim.Act.Achieve

NOTES

Fundamentals of Java - 6

Introduction to Object Oriented Principels Creating Class and Objects

Introduction to Object-Oriented Principles (OOP)

Object-Oriented Programming (OOP) is a way of building software that is inspired by real-world things, like people or machines. Just like these real-world objects have parts and actions they can do, OOP allows us to create "objects" in our code with their own data and behaviours. It also helps us show how these objects relate to each other, just like relationships between things in real life.

The Four Pillars of OOP

1. **Inheritance: Reusing Traits**
   Inheritance lets new objects reuse properties of existing ones.
   Example: A "Vehicle" can be the parent; "Car" and "Bike" inherit “can move” from it.
2. **Polymorphism: Acting in Different Ways**
   Polymorphism means an object can behave differently based on the situation.
   Example: A teacher is an educator at school but a parent at home.
3. **Encapsulation: Hiding the Details**
   Encapsulation hides internal parts, only showing what’s needed.
   Example: A bank account shows balance through deposit and withdrawal but hides the exact amount.
4. **Abstraction: Focusing on What Matters**
   Abstraction shows essential details, hiding complexity.
   Example: Driving a car only involves the wheel and pedals, without needing to know the engine’s workings.

Creating Class and Objects

Before creating class and object let’s understand

What is a Class ?

A class is like a blueprint or template:
- Like a **cookie cutter** that makes cookies
- Like a **house plan** that builders use
- Like a **car design** that factories use

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:
Fields
Methods
Constructors
Blocks
Nested class
Interface

Syntax to declare a class:

```
Java
class <class_name> {
 Field;
 method;
}
```

Creating a Simple Class Example:

```
Java
public class Car {
    // Properties (also called attributes)
    String brand;
```

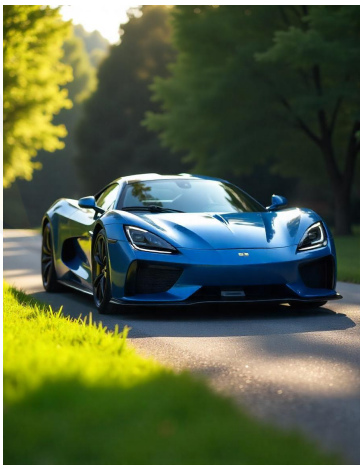
```
    String color;
    int year;

    // Method
    public void startEngine() {
        System.out.println("The car is starting");
    }

    public void brake() {
        System.out.println("The car is stopping");
    }
}
```

What is an Object ?

An object is something made from a class:
- Like a **cookie made** from a cookie cutter
- Like a **house built** from house plans
- Like a **car made** from car design



An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.

An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc. It can be physical or logical (tangible and intangible).

The example of an intangible object is the banking system.

An object has three characteristics:

State: represents the data (value) of an object.

Behaviour: represents the behaviour (functionality) of an object such as deposit, withdraw, etc.

Identity: An object identity is typically implemented via a unique ID. The the value of the ID is not visible to the external user. However, it is used internally by the JVM to identify each object uniquely.

For **Example**, Pen is an object. Its name is Reynolds; color is white, known as its state. It is used to write, so writing is its behavior.

An object is an instance of a class. A class is a template or blueprint from which objects are created. So, an object is the instance(result) of a class.

Object Definitions:

An object is a real-world entity.

An object is a runtime entity.

The object is an entity which has state and behavior.

The object is an instance of a class.

Creating Objects (Called Instantiation):

// Making a new car object

```java
Car myCar = new Car();
myCar.brand = "Toyota";
myCar.color = "Red";
myCar.year = 2024;
```

// Using the car

```java
myCar.startEngine();  // Output: The car is starting
```

Real World Examples

Example 1:

//Creating Student Class

```java
public class Student {
    // Properties
    String name;
    int age;
    String grade;

    // Methods
    public void study() {
        System.out.println(name + " is studying");
    }

    public void submitHomework() {
        System.out.println(name + " submitted homework");
    }
}
```

// Creating student objects

```java
Student john = new Student();
john.name = "John";
john.age = 15;
john.grade = "10th";

Student mary = new Student();
mary.name = "Mary";
mary.age = 16;
mary.grade = "11th";
```

Example 2:

//Creating BankAccount Class

```java
public class BankAccount {
    // Properties
    String accountHolder;
    double balance;

    // Methods
    public void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: $" + amount);
        System.out.println("New balance: $" + balance);
    }

    public void withdraw(double amount) {
        if(amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: $" + amount);
            System.out.println("New balance: $" + balance);
        } else {
            System.out.println("Not enough money!");
        }
    }
}
```

// Using BankAccount

```java
BankAccount myAccount = new BankAccount();
myAccount.accountHolder = "John Smith";
myAccount.balance = 1000.0;

myAccount.deposit(500.0);   // Adds money
myAccount.withdraw(200.0);  // Takes out money
```

Important OOP Concepts

Constructor - The Builder
(We will Cover this in details in upcoming Lecture)

Like setting up a new phone when you first buy it:

```java
public class Phone {
    String brand;
    String model;

    // Constructor
    public Phone(String phoneBrand, String phoneModel) {
        brand = phoneBrand;
        model = phoneModel;
    }
}

// Using constructor
Phone myPhone = new Phone("Apple", "iPhone 15");
```

Methods - The Actions

(We will Cover this in details in upcoming Lecture)

Like buttons on a remote control:

```java
public class Television {
    boolean isOn = false;

    public void turnOn() {
        isOn = true;
        System.out.println("TV is now ON");
    }

    public void turnOff() {
        isOn = false;
        System.out.println("TV is now OFF");
    }
}
```

Practice Example: Complete Class

Game Character Example:

```java
public class GameCharacter {
    // Properties
    String name;
    int health;
    String weapon;

    // Constructor
    public GameCharacter(String characterName) {
        name = characterName;
        health = 100;  // Start with full health
        weapon = "None";
    }

    // Methods
    public void pickupWeapon(String newWeapon) {
        weapon = newWeapon;
        System.out.println(name + " picked up " + weapon);
    }

    public void takeDamage(int damage) {
        health = health - damage;
        if(health < 0) health = 0;
        System.out.println(name + "'s health: " + health);
    }

    public void heal(int amount) {
        health = health + amount;
        if(health > 100) health = 100;
        System.out.println(name + "'s health: " + health);
    }
}
```

// Using the GameCharacter

```java
GameCharacter player1 = new GameCharacter("Hero");
player1.pickupWeapon("Sword");     // Hero picked up Sword
player1.takeDamage(20);            // Hero's health: 80
player1.heal(10);                  // Hero's health: 90
```

Final Brief on Class and Object

1. Class is the blueprint (like recipe)
2. Object is the thing made (like cooked food)
3. Properties are characteristics (like color, size)
4. Methods are actions (like start, stop)
5. Constructor sets up new objects (like setup instructions)

Common Mistakes to Avoid

1. Don't forget to create object before using it
2. Don't confuse class name and object name
3. Remember class names start with capital letter
4. Remember to use 'new' keyword when creating objects

Finally: Classes and Objects make our code organized and reusable!

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan